

Hands-on Activity 5.1

Player Control

Course Code: CPE 411

Program: Computer Engineering

Course Title: Interactive Extended Reality

Date Performed: 19/03/2026

Section: 22S2

Date Submitted: 19/03/2026

Name(s): Ross Andrew Bulaong

Instructor: Engr. Jimlord M. Quejado

1. Discussion

Learn how to create new C# scripts, call basic functions, and use variables to modify the results of those functions. In this activity you'll translate keyboard, mouse, or game-pad input into smooth on-screen motion. You'll explore Unity's Input System, learn how to read player actions every frame, and apply those inputs to transform velocity, rotation, and animation states. By the end, you'll have a fully controllable avatar that feels responsive and fun—an essential foundation for every game you'll build from here on.

2. Materials and Equipment

Unity Hub and Unity Editor

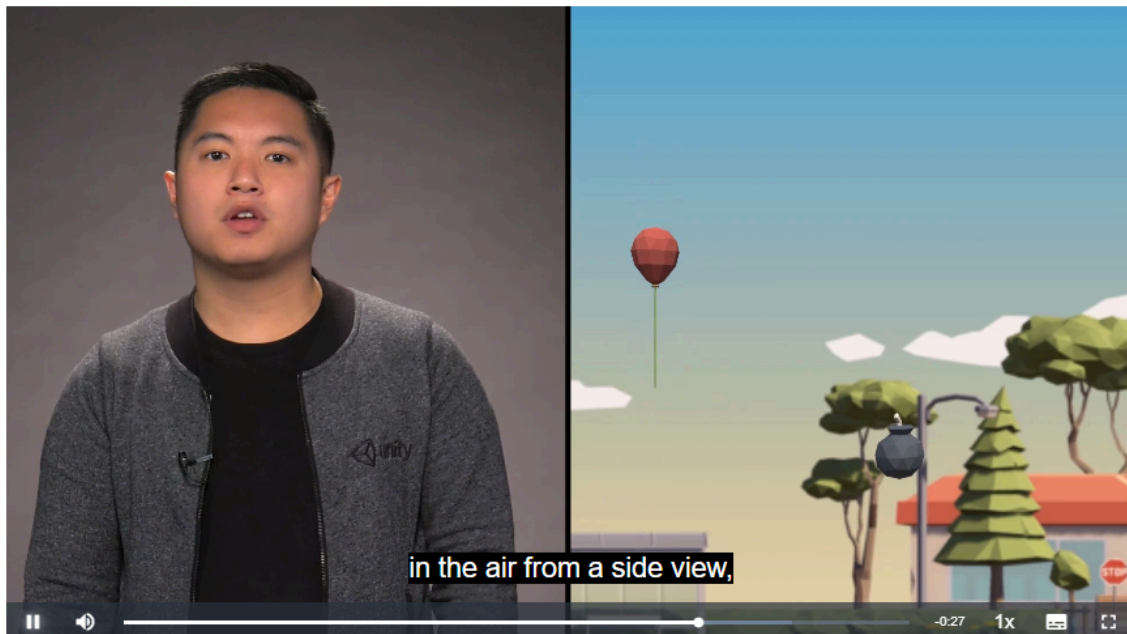
Learn Unity Website ([Unity Essentials Pathway - Learn Game Development for Beginners | Unity Learn](#))

Computer with Network Connection

3. Procedure

1. Unit 1 - Introduction

 Q&A (0)



Mark Step As Incomplete

Watch the introduction

Universal 3D

The URP (Universal Render Pipeline) blank template includes the settings and assets you need to start creating with URP. Equipped with artist-...

 Read more  Upgrade

Unity organization

RossUsesUnity

Project name *

Prototype 1

Location *

C:\Users\Andrew Bulaong\Create With ...

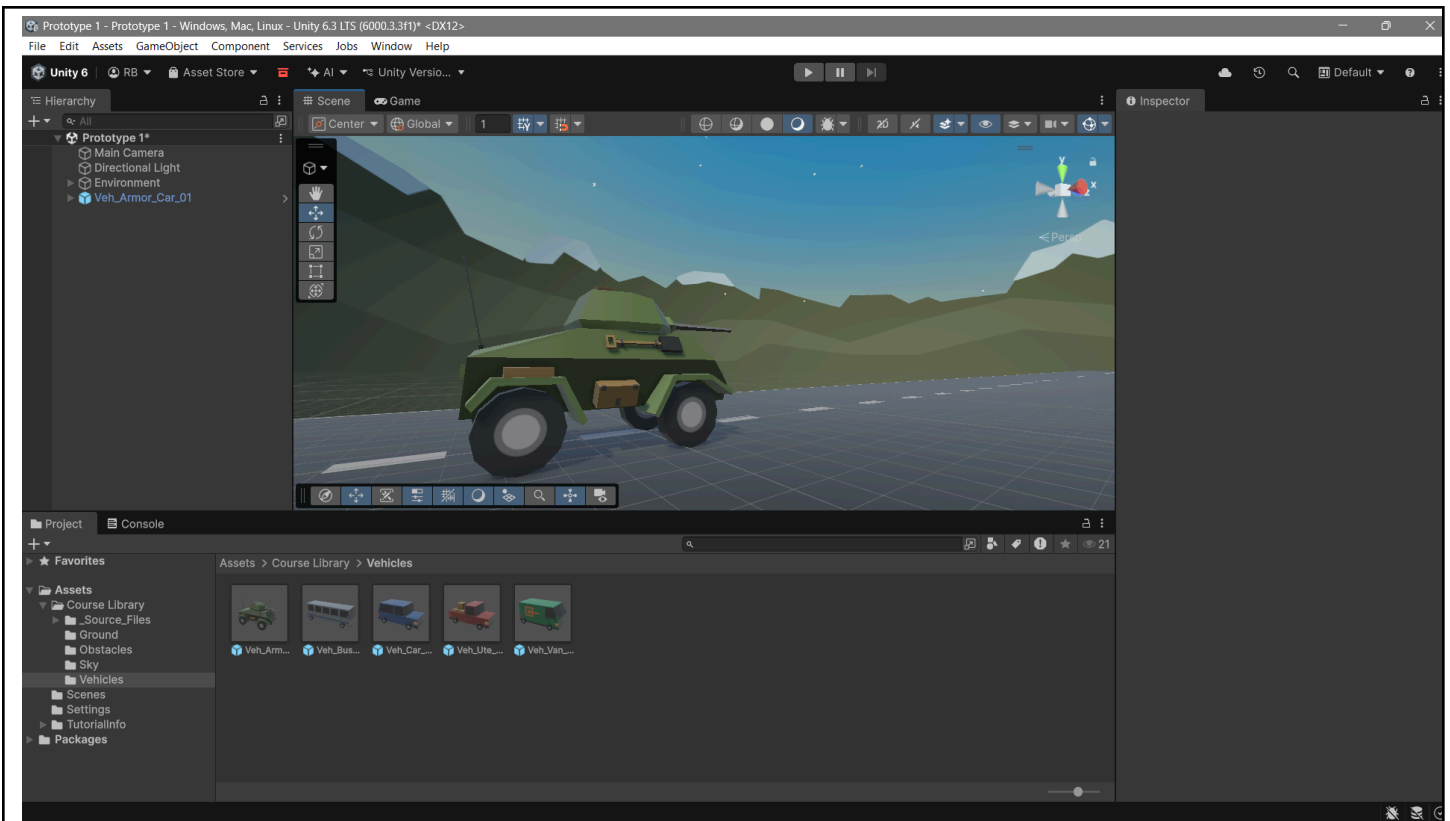
☐ Use Unity Version Control

+ Create project

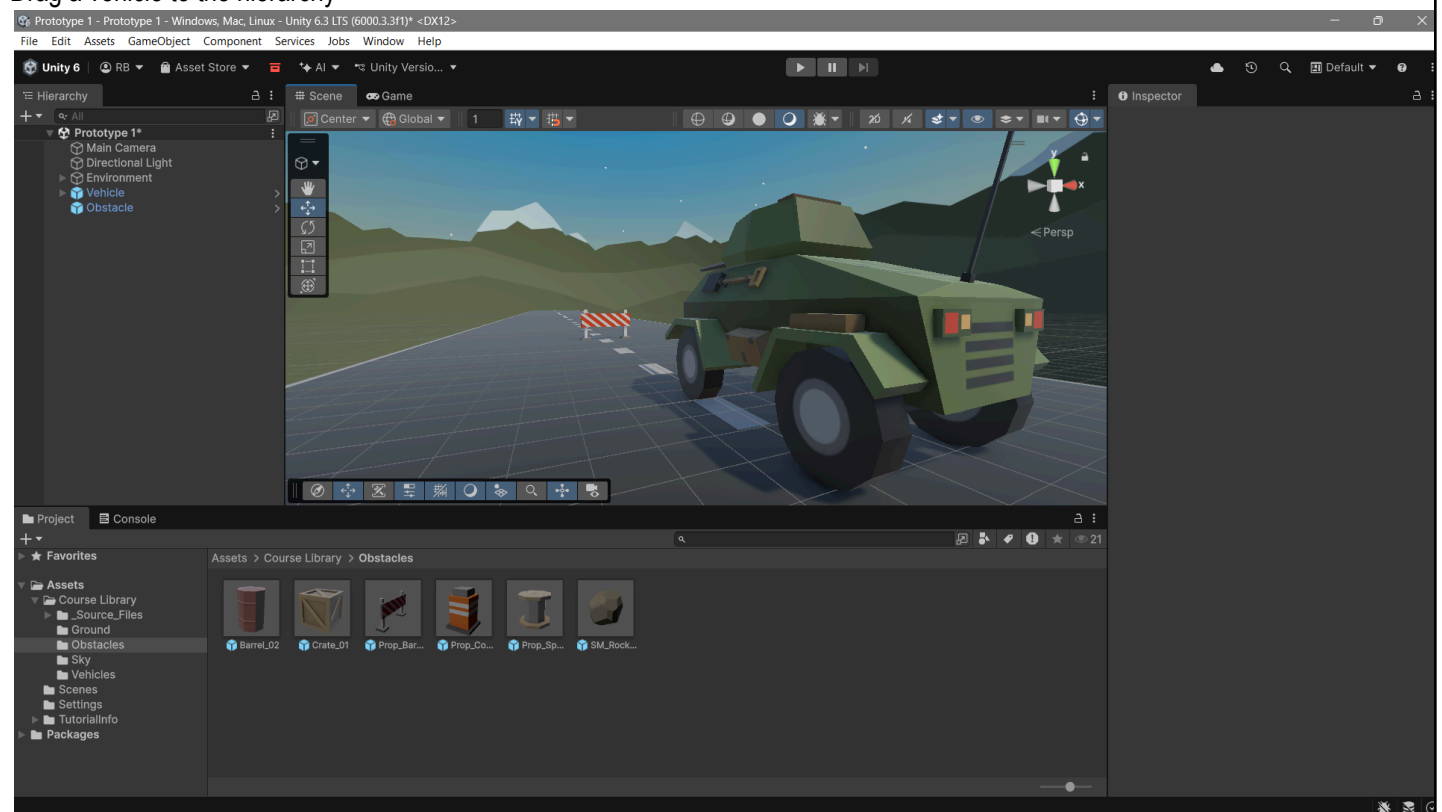
Create a new project for this laboratory, in the Create With Code folder instructed to be made

Name	Date modified	Type	Size
 Prototype-1_Starter-Files	09/09/2021 9:42 am	Unity package file	764 KB

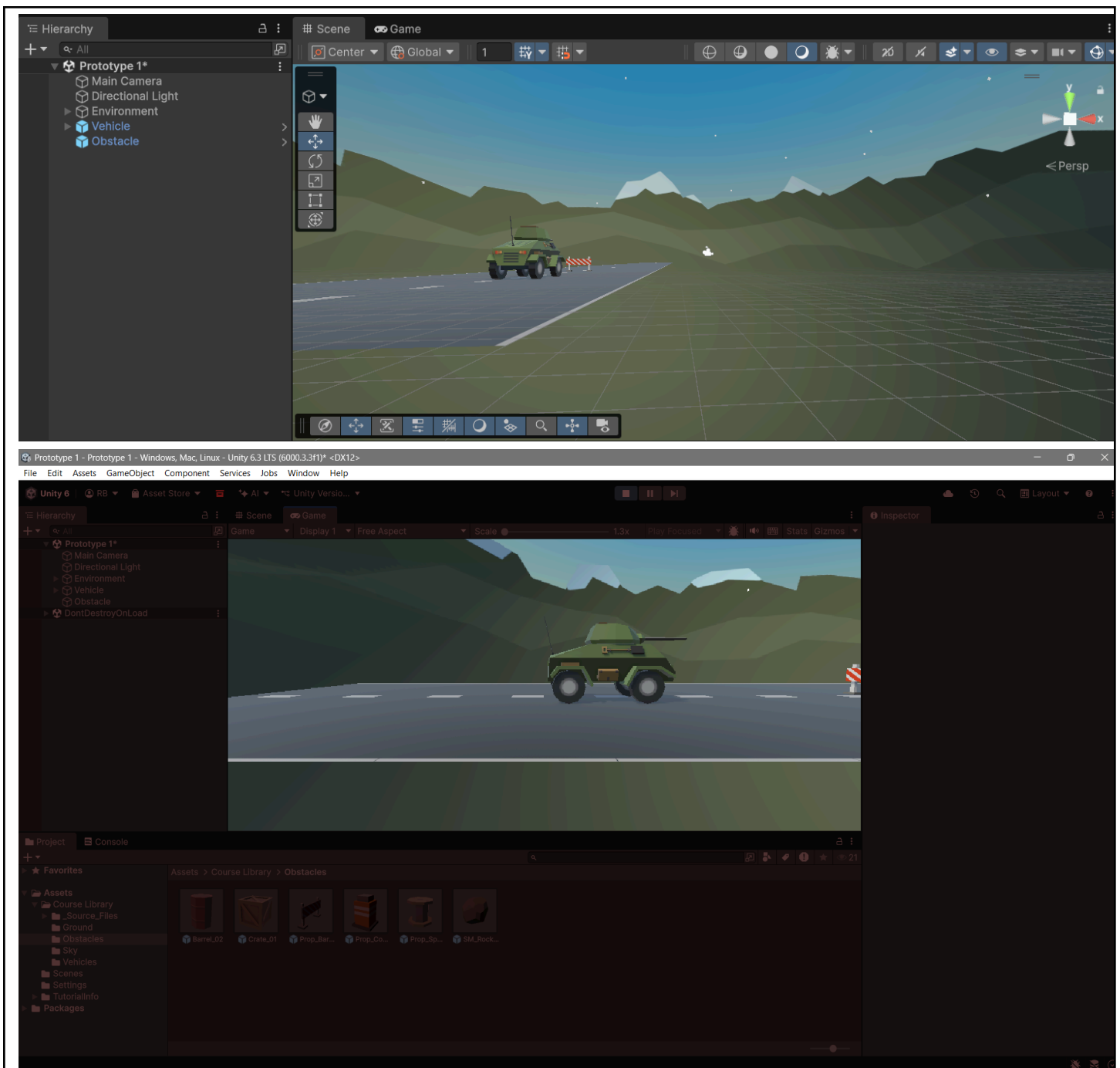
Import the provided package to get the assets



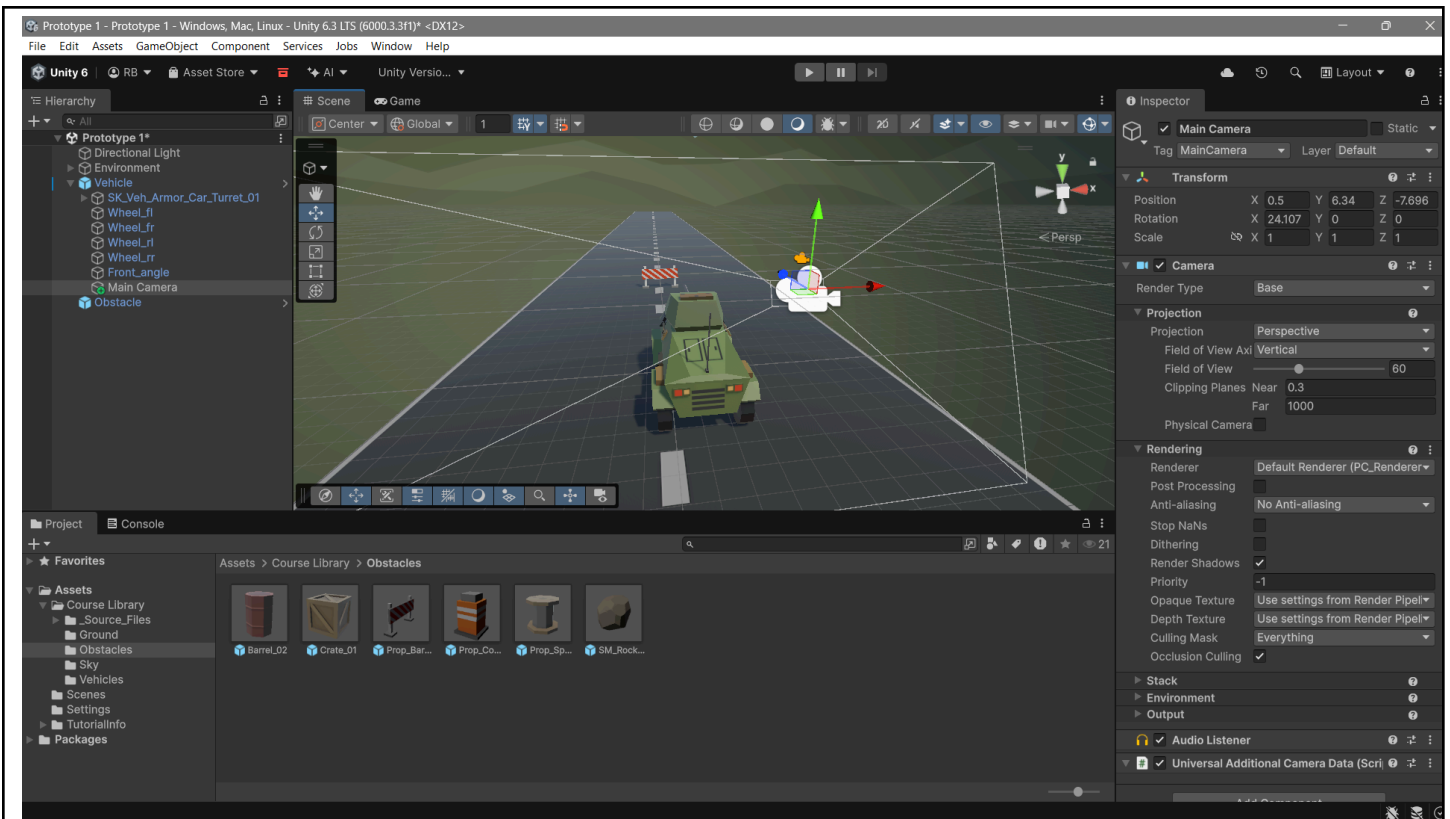
Drag a vehicle to the hierarchy



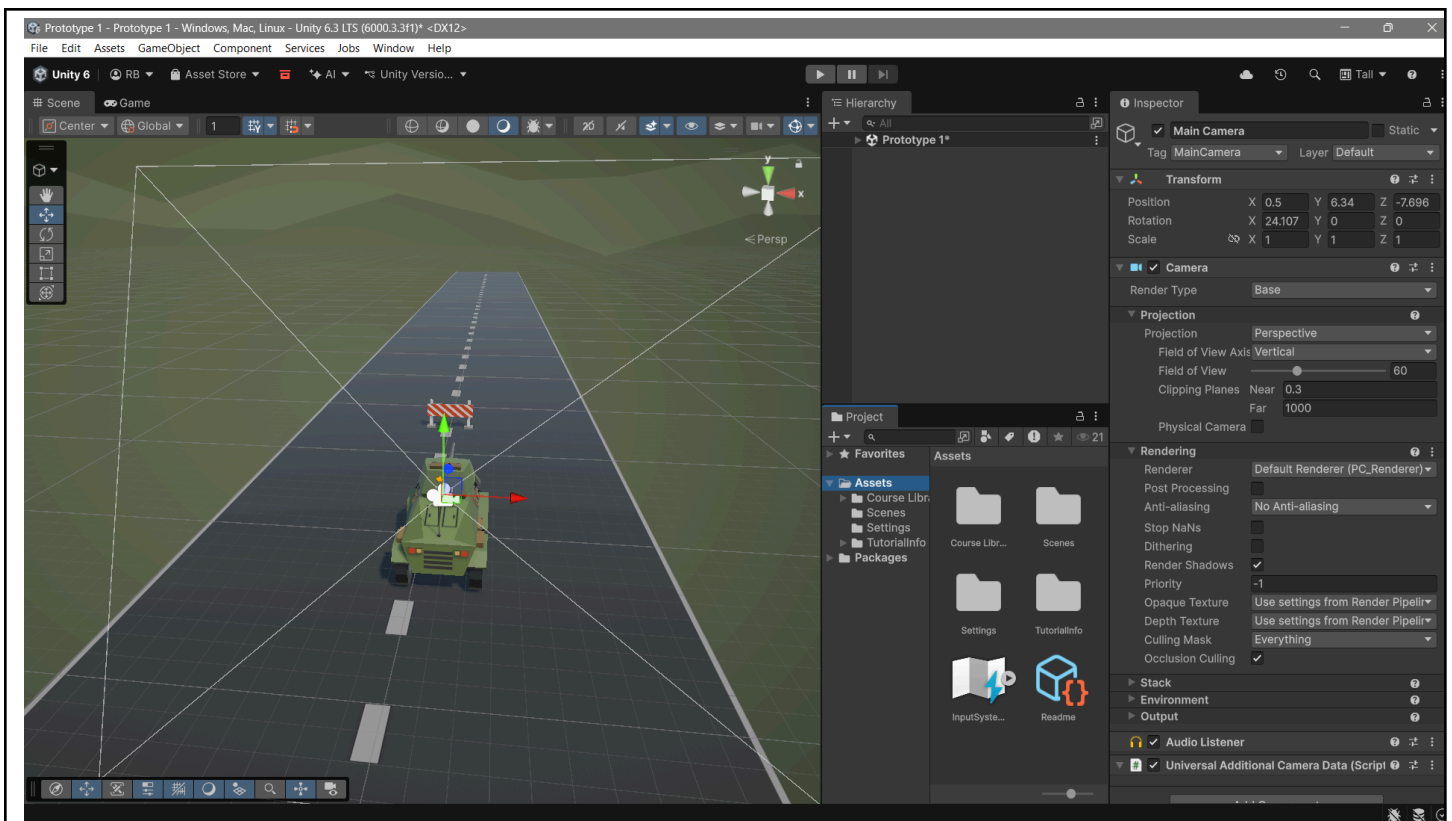
Add obstacle, rename assets



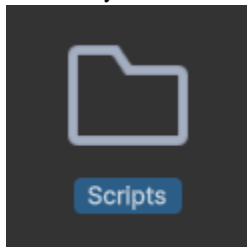
Frame the camera and test the game; camera position is awkward



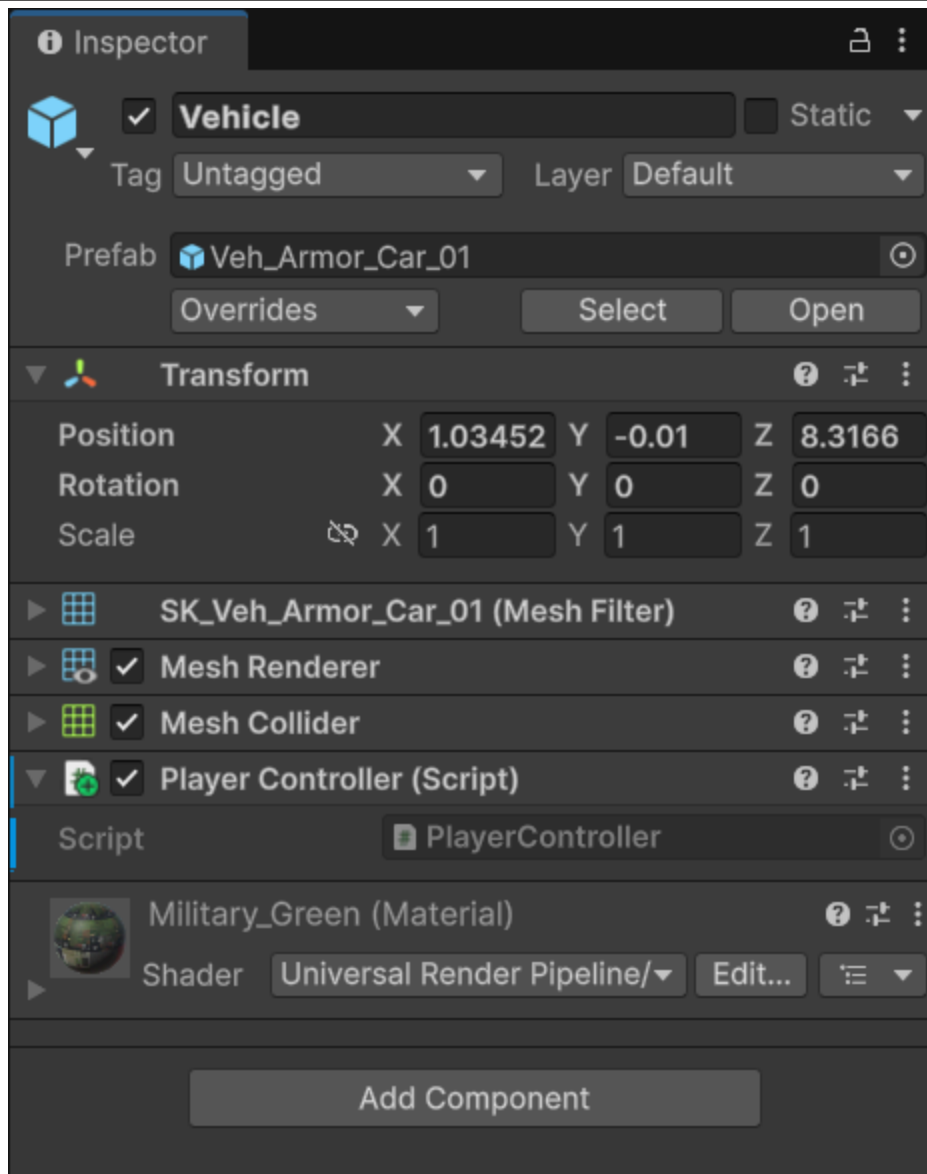
Rotate the camera to a better position, i personally added it to the Vehicle gameObject as a child so it moves with the vehicle



Set the layout to tall for better visualization



Create script folder



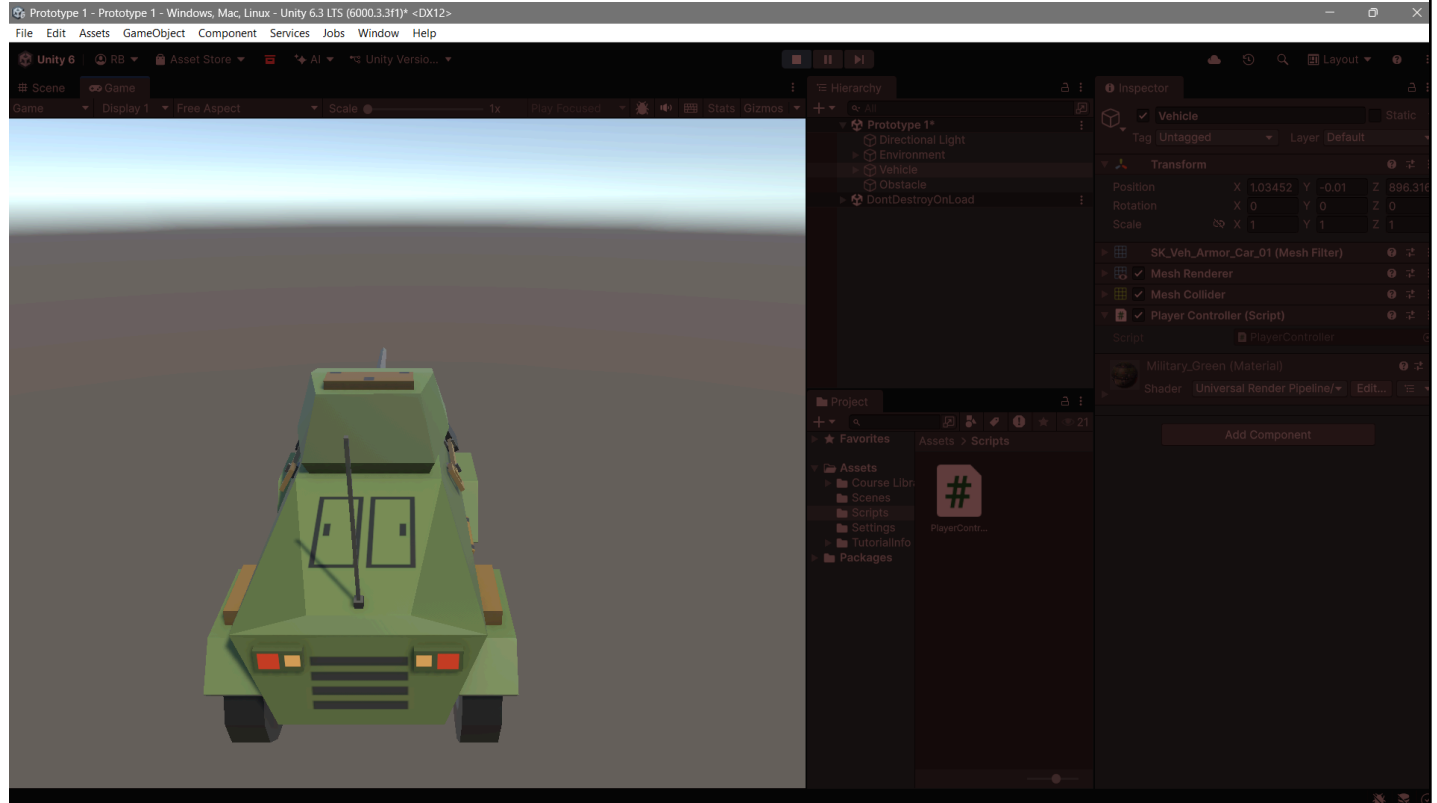
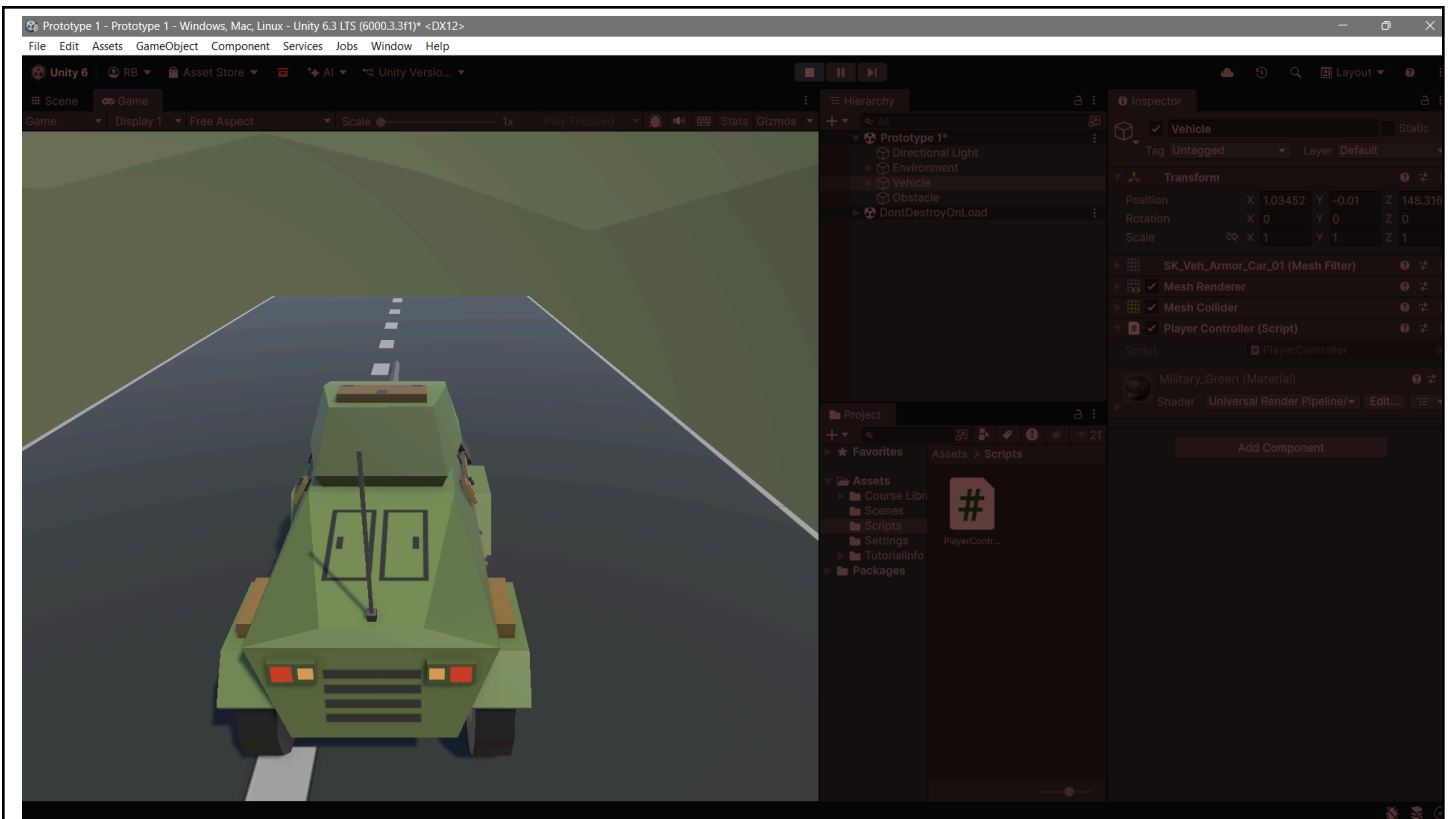
Create a monobehaviour script called PlayerController, add it to the vehicle

```
// Update is called once per frame
void Update()
{
    // Move the vehicle forward per frame
}
```

Add a comment to the Update function that runs every frame

```
// Update is called once per frame
void Update()
{
    // Move the vehicle forward per frame
    transform.Translate(0,0,1);
}
```

Move the vehicle forward by transforming its position 1 unit per frame



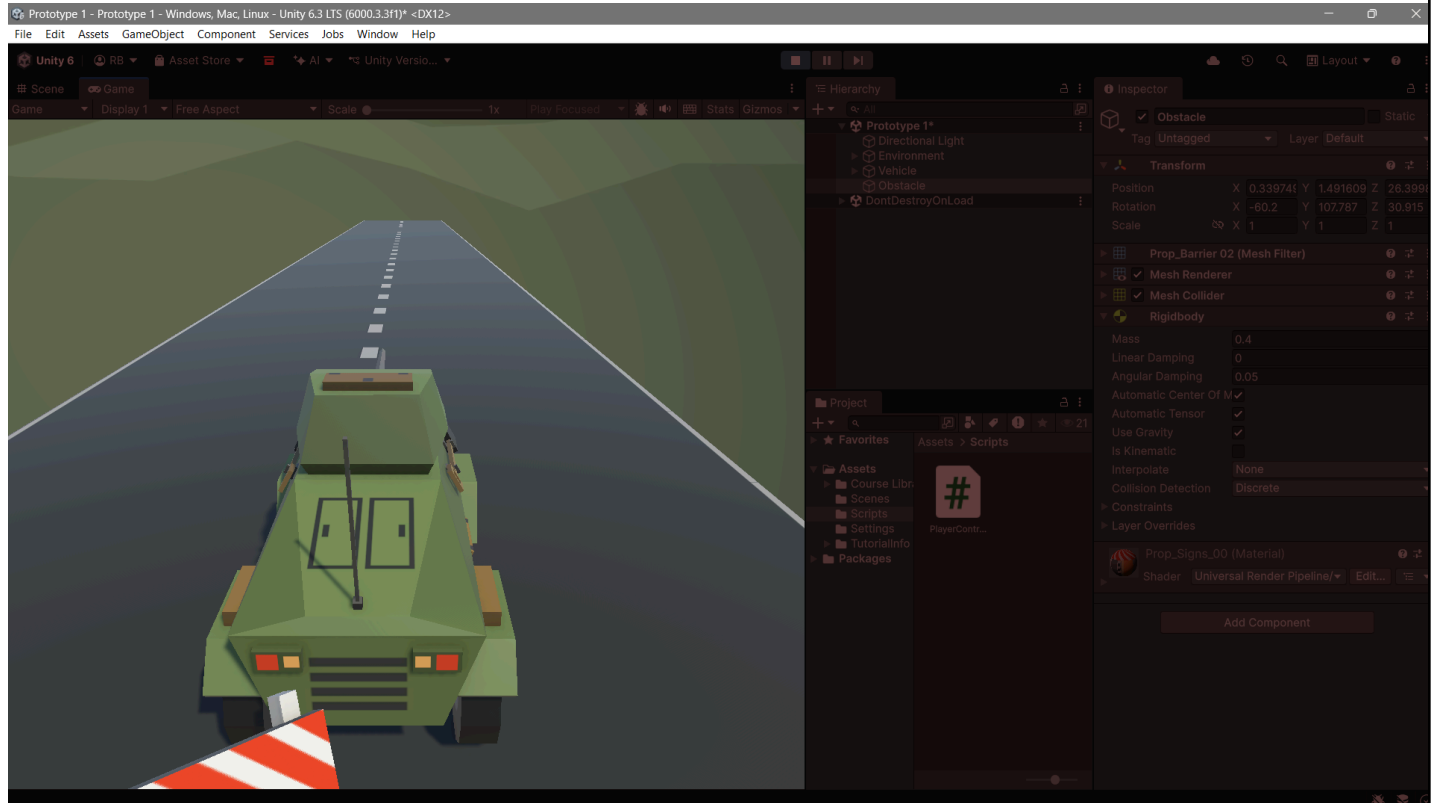
As I test the script's functionality, the car moves 1 unit per frame, and is now outside of the map.

```
// Update is called once per frame
void Update()
{
    // Move the vehicle forward per frame
    transform.Translate(Vector3.forward);
}
```

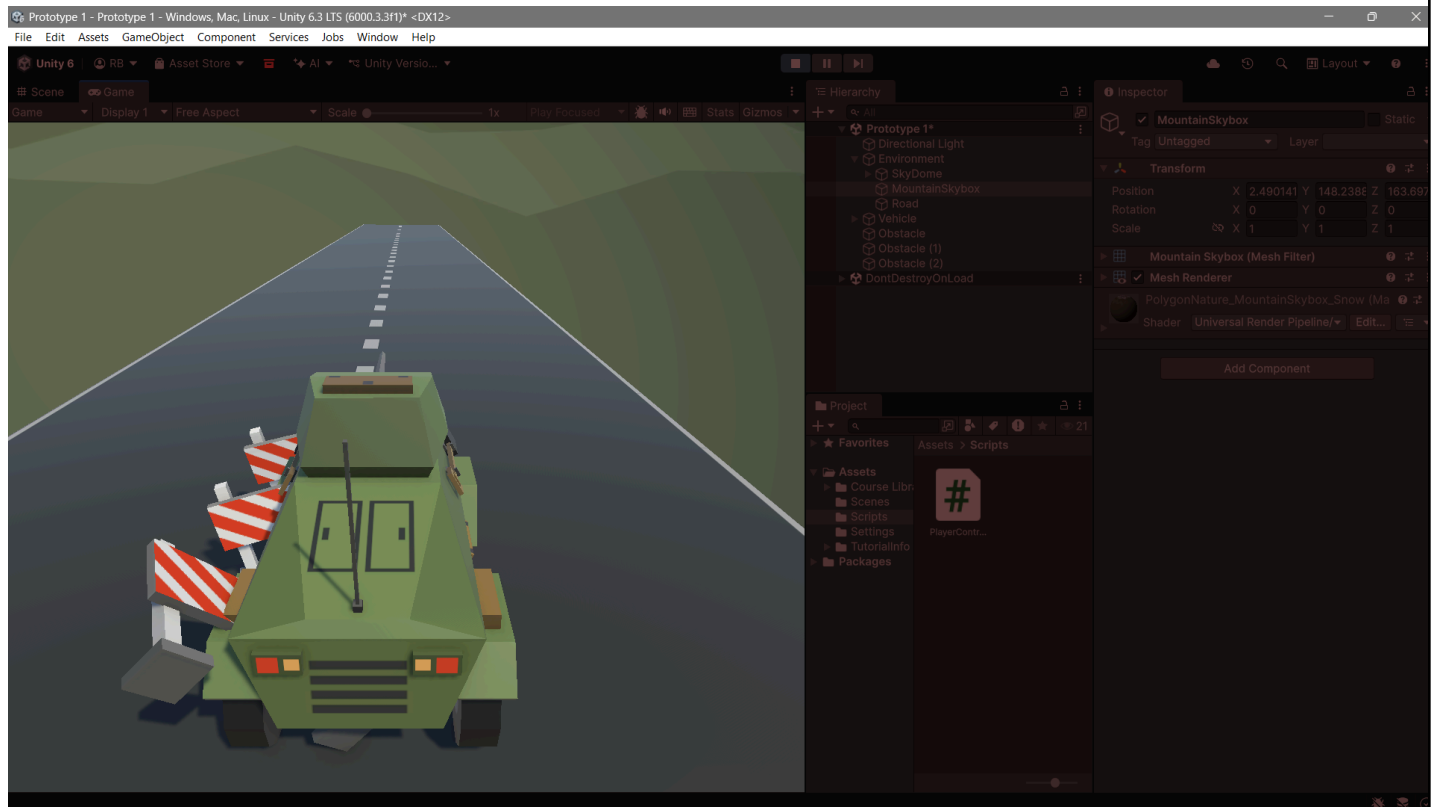
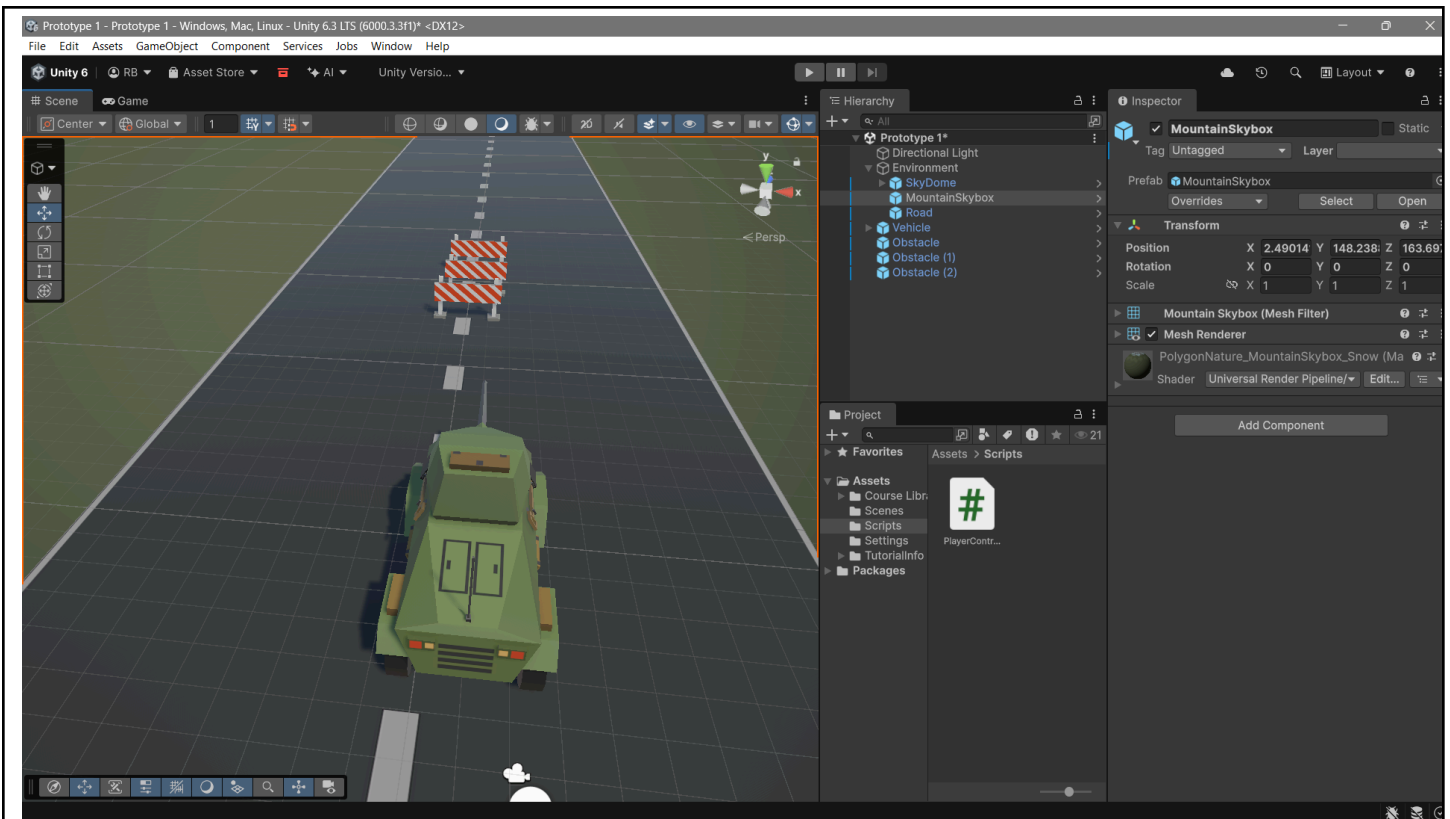

Use a cleaner version of the script by using `Vector3.forward` rather than `(0,0,1)` coordinates. Works exactly like last time, but cleaner.

```
// Update is called once per frame
void Update()
{
    // Move the vehicle forward per frame
    transform.Translate(Vector3.forward * Time.deltaTime * 20);
}
```

Adjust the player's speed by utilizing delta time, the speed that returns the interval between the previous frame and the next one; ensuring that gameplay / objects move relatively the same regardless of how many frames are being run per second. without it, since `vector3.forward` runs per frame, a 300 fps game would run that same command 300 times in a second, accelerating the object far too much for the intended purpose. Adjusting the constant 20 changes the speed of the vehicle. (I've learned about this in my prior 2D unity game i've build myself)



Added a rigidbody to both the vehicle and the obstacle, then changed the masses of both. The two objects now interact with one another.



Added more objects to the scene. Will reposition these as the tutorial progresses with lateral movement, allowing dodging; which is essentially an obstacle course.

```

public float speed = 20.0f;
// Update is called once per frame
void Update()
{
    // Move the vehicle forward per frame
    transform.Translate(Vector3.forward * Time.deltaTime * speed);
}

```

Added a publicly available speed variable (editable via inspector) instead of a constant number set in the code editor. default is 20.0f as the previous speed was 20, but its editable so adjustments can be made anyways

```

1  using UnityEngine;
2
3
4  public class FollowPlayer : MonoBehaviour
5  {
6      public GameObject player;
7
8      // Start is called once before the first execution of Update after the MonoBehaviour is created
9      void Start()
10     {
11
12     }
13
14
15     // Update is called once per frame
16     void Update()
17     {
18         transform.position = player.transform.position;
19     }
20 }
21

```

Create a script that follows the player (although attaching the MainCamera to the object works, the tutorial has other methods which I will follow, ofcourse). which starts by declaring a public GameObject variable called player assigned in the inspector, which the script then uses as a reference to this script's Update function where it moves the camera's position to the position of the player GameObject. After testing, the camera sees the vehicle's wheels.

```

// Update is called once per frame
void Update()
{
    transform.position = player.transform.position + new Vector3 (0,5,-7);
}

```

Add a new Vector to the transform.position vector to create a new vector position so that the camera isn't laying flat on the player object's (0,0,0) position, which it previously laid, causing the camera to be directly in the vehicle's wheels. using the new constructor initializes new values for the Vector3 data type, as without it, an error will occur.

```

1  using UnityEngine;
2
3
4  public class FollowPlayer : MonoBehaviour
5  {
6      public GameObject player;
7      private offset = Vector3(0,5,-7)
8
9      // Start is called once before the first execution of Update after the MonoBehaviour is created
10     void Start()
11     {
12
13     }
14
15     // Update is called once per frame
16     void Update()
17     {
18         transform.position = player.transform.position + offset;
19     }
20 }
21

```

create a private (ineditable, unseen by inspector) variable to be added to the transform.position statement instead of hardcoding the number directly. The Private access modifier safeguards a variable from external changes.

```

1  using UnityEngine;
2
3
4  public class FollowPlayer : MonoBehaviour
5  {
6      public GameObject player;
7      private offset = Vector3(0,5,-7)
8
9      void LateUpdate()
10     {
11         transform.position = player.transform.position + offset;
12     }
13 }
14

```

Change the Update method to LateUpdate() which ensures that the update of the camera in terms of the vehicle executes after all the update methods have been called. This ensures that the vehicle moves first before the camera tracks it / repositions itself.

```

1  using UnityEngine;
2
3  public class PlayerController : MonoBehaviour
4  {
5      // Start is called once before the first execution of Update after the MonoBehaviour is created
6      void Start()
7      {
8
9      }
10
11     public float speed = 20.0f;
12     public float turnSpeed;
13     // Update is called once per frame
14     void Update()
15     {
16         // Move the vehicle forward per frame
17         transform.Translate(Vector3.forward * Time.deltaTime * speed);
18         transform.Translate(Vector3.right * Time.deltaTime * turnSpeed);
19     }
20 }
21

```

Adding a public turnSpeed variable that controls the speed of the vehicle's turn (left / right)

```

public float speed = 20.0f;
public float turnSpeed;
public float horizontalInput;

// Update is called once per frame
void Update()
{
    horizontalInput = Input.GetAxis("Horizontal");

    // Move the vehicle forward per frame
    transform.Translate(Vector3.forward * Time.deltaTime * speed);
    // Moves the vehicle right or left by multiplying it to the horizontalInput values (if there are no horizontalInput values, it's set to 0 and therefore it'll just move straight)
    transform.Translate(Vector3.right * Time.deltaTime * turnSpeed * horizontalInput);
}

```

Add a horizontalInput variable multiplier to the transform.Translate runs every frame to create horizontal movement. Essentially, in this code, it does two things per frame: move forward, if there is horizontal input (left / right) and if true, adjusts the object's position based on the horizontalInput values.

```

10
11     public float speed = 20.0f;
12     public float turnSpeed;
13     private float horizontalInput;
14     private float verticalInput;
15
16     // Update is called once per frame
17     void Update()
18     {
19         // Gets the horizontal input values
20         horizontalInput = Input.GetAxis("Horizontal");
21         // Gets the vertical input values
22         verticalInput = Input.GetAxis("Vertical");
23
24         // Move the vehicle forward per frame
25         transform.Translate(Vector3.forward * Time.deltaTime * speed * verticalInput);
26         // Moves the vehicle right or left by multiplying it to the horizontalInput values (if there are no horizontalInput values, it's set to 0 and therefore it'll just move straight)
27         transform.Translate(Vector3.right * Time.deltaTime * turnSpeed * horizontalInput);
28     }
29 }
30

```

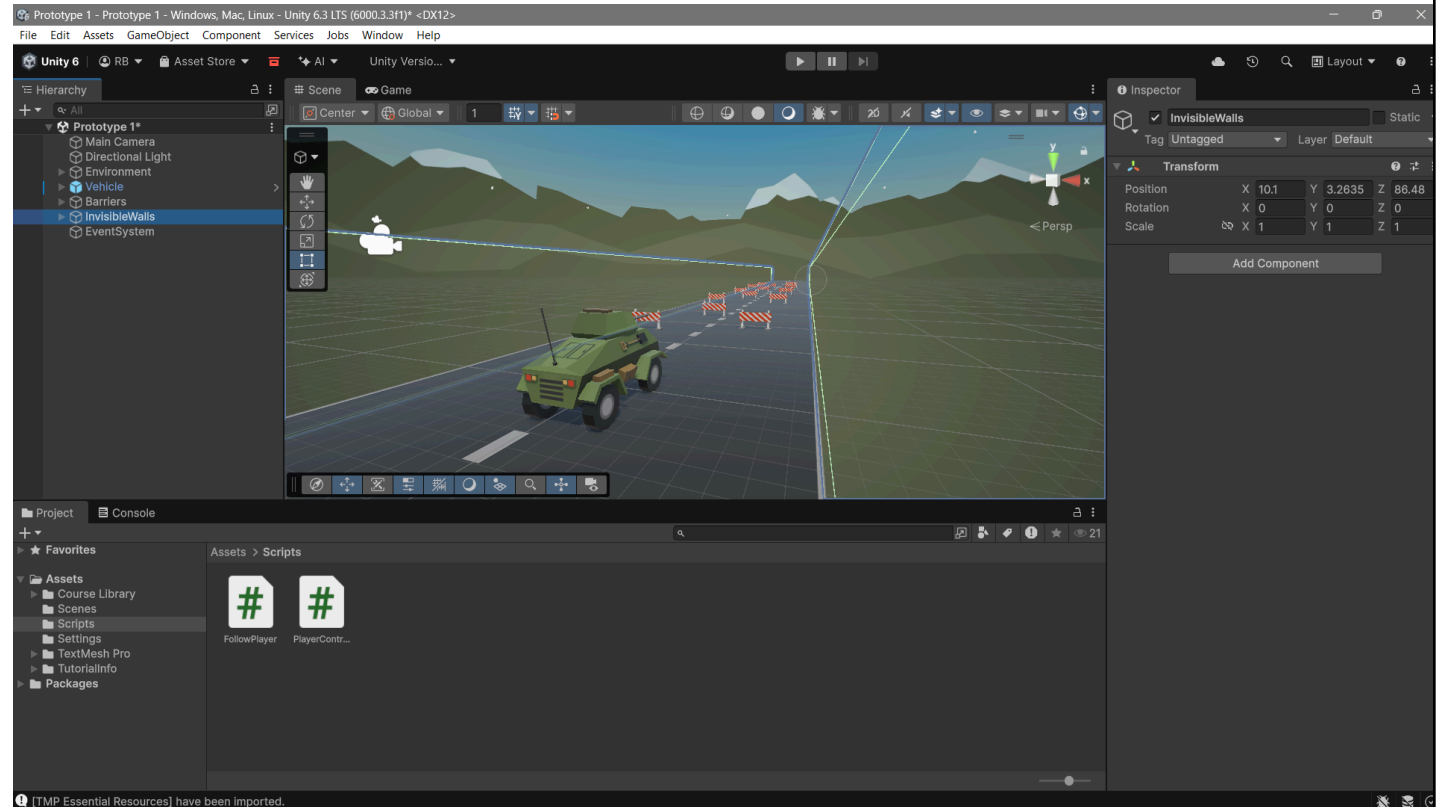
Add a verticalInput variable and multiply it to the statement that makes the vehicle go forward per frame; essentially making the forward and backward movement (after adding the verticalInput) dependent on the player; as without input, the value of Transform.translate becomes 0 and therefore does not move.

```

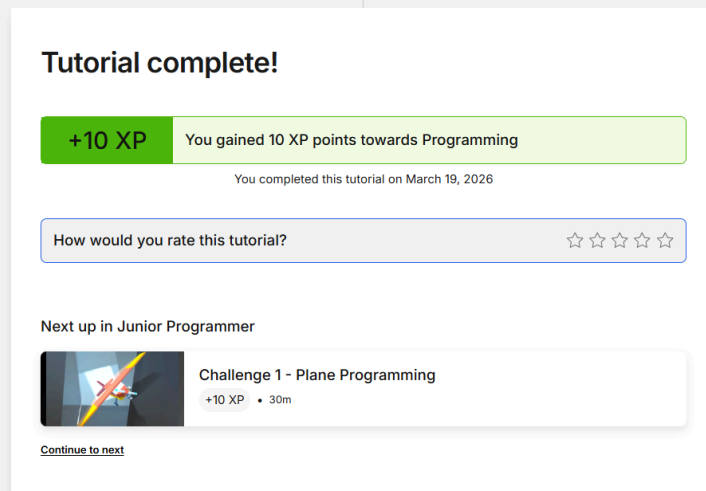
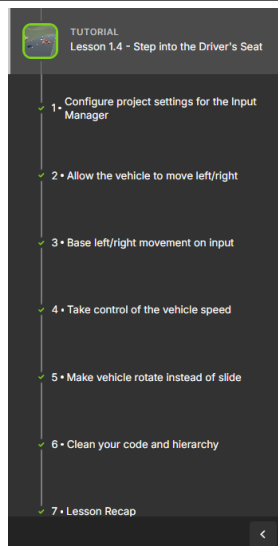
11 public float speed = 20.0f;
12 public float turnSpeed;
13 private float horizontalInput;
14 private float verticalInput;
15
16 // Update is called once per frame
17 void Update()
18 {
19     // Gets the horizontal input values
20     horizontalInput = Input.GetAxis("Horizontal");
21     // Gets the vertical input values
22     verticalInput = Input.GetAxis("Vertical");
23     // Move the vehicle forward per frame
24     transform.Translate(Vector3.forward * Time.deltaTime * speed * verticalInput);
25     // Moves the vehicle right or left by multiplying it to the horizontalInput values (if there are no horizontalInput values, it's set to 0 and therefore it'll just move straight)
26     transform.Rotate(Vector3.up * Time.deltaTime * turnSpeed * horizontalInput);
27 }

```

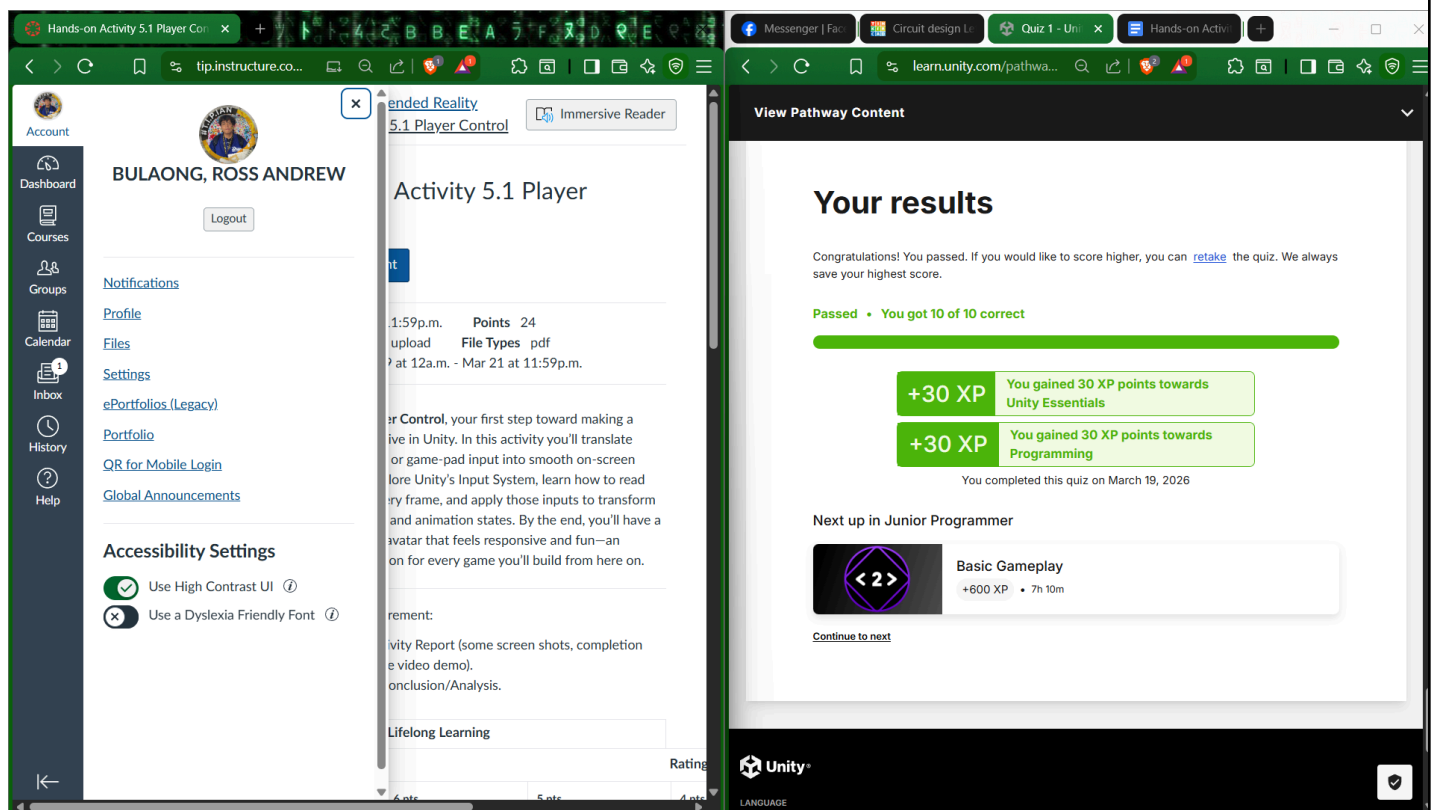
Change the horizontal movement to rotating towards left to right instead of just moving. This creates a better effect much similar to actual IRL vehicles. I also cleaned up the code as instructed; but I kept the speed and turnSpeed public for easier testing developer-side.



Personal adjustments I've made: I added more barricades, created invisible walls so the player doesn't phase through before they reach the end, adjusted the mass of the vehicle to 200, and the turn speed to 50.



Completed the Unity tutorial up to 1.4 as instructed



Quiz results

DEMONSTRATION LINK: [PlayerControl_BULAONG_DEMO.mp4](#)

4. Conclusion

Even though I am already familiar with Unity scripting as I've made my own game before, it is still a wonderful, educational experience for me. I've learned about `LateUpdate()` which I've never used before. This activity solidified my understanding of Unity scripting, and gave me an activity to apply the skills I've learned along the way.

5. Assessment Rubric

Lab Activity Rubric



Criteria	Ratings						Pts
SO 7 PI 1 Student Outcome 7.1 Acquire and apply new knowledge from outside sources. threshold: 4.8 pts	6 pts Excellent Educational interests and pursuits exist and flourish outside classroom requirements, knowledge and/or experiences are pursued independently and applies knowledge learned into practice	5 pts Good Educational interests and pursuits exist and flourish outside classroom requirements, knowledge and/or experiences are pursued independently	4 pts Satisfactory Look beyond classroom requirements, showing interest in pursuing knowledge independently	3 pts Unsatisfactory Begins to look beyond classroom requirements, showing interest in pursuing knowledge independently	2 pts Poor Relies on classroom instruction only	1 pts Very Poor No initiative or interest in acquiring new knowledge	6 pts
SO 7 PI 2 Student Outcome 7.2 Learn independently threshold: 4.8 pts	6 pts Excellent Completes an assigned task independently and practices continuous improvement	5 pts Good Completes an assigned task without supervision or guidance	4 pts Satisfactory Requires minimal guidance to complete an assigned task	3 pts Unsatisfactory Requires detailed or step-by-step instructions to complete a task	2 pts Poor Shows little interest to complete a task independently	1 pts Very Poor No interest to complete a task independently	6 pts
SO 7 PI 3 Student Outcome 7.3 Critical thinking in the broadest context of technological change threshold: 4.8 pts	6 pts Excellent Synthesizes and integrates information from a variety of sources; formulates a clear and precise perspective; draws appropriate conclusions	5 pts Good Evaluate information from a variety of sources; formulates a clear and precise perspective.	4 pts Satisfactory Analyze information from a variety of sources; formulates a clear and precise perspective.	3 pts Unsatisfactory Apply the gathered information to formulate the problem	2 pts Poor Gather and summarized the information from a variety of sources but failed to formulate the problem	1 pts Very Poor Gather information from a variety of sources	6 pts
SO 7 PI 4 Student Outcome 7.4 Creativity and adaptability to new and emerging technologies threshold: 4.8 pts	6 pts Excellent Ideas are combined in original and creative ways in line with the new and emerging technology trends to solve a problem or address an issue.	5 pts Good Ideas are creative and adapt the new knowledge to solve a problem or address an issue	4 pts Satisfactory Ideas are creative in solving a problem, or address an issue	3 pts Unsatisfactory Shows some creative ways to solve the problem	2 pts Poor Shows initiative and attempt to develop creative ideas to solve the problem	1 pts Very Poor Ideas are copied or restated from the sources consulted	6 pts

Total Points: 24